Джанго - это специальный инструмент, который помогает разработчикам создавать веб-сайты и приложения. Он называется фреймворком, потому что предоставляет готовые инструменты и правила для работы с веб-сайтами.

Джанго нужен для того, чтобы упростить процесс разработки веб-сайта. Вместо того, чтобы создавать каждую часть сайта с нуля, разработчики могут использовать Джанго, чтобы получить уже готовые компоненты и функциональность.

Джанго имеет много полезных функций, которые помогают создавать веб-сайты:

- Он позволяет создавать красивые и интерактивные страницы, которые пользователи могут видеть в своих браузерах.
- Он облегчает работу с базой данных, где хранится информация о веб-сайте, такая как список товаров или информация о пользователях.
- Он помогает обрабатывать запросы от пользователей и отправлять им нужную информацию.
- Он имеет мощный административный интерфейс, который позволяет администраторам управлять содержимым веб-сайта без необходимости знать программирование.

Таким образом, Джанго помогает разработчикам создавать веб-сайты более быстро и эффективно, предоставляя им готовые инструменты и структуру работы. Он широко используется для создания различных веб-сайтов, таких как блоги, интернет-магазины и социальные сети.

От запроса до отрисовки страницы

Вот основные этапы и внутренние действия, которые происходят при обработке запроса и отрисовке страницы в Django:

1. Пользователь открывает браузер и вводит адрес веб-сайта (URL) в строке поиска.

2. Браузер отправляет запрос на сервер, где расположен веб-сайт.
3. Сервер получает этот запрос и передает его в Django.
4. Django получает запрос и смотрит на URL, чтобы понять, какая страница должна быть показана.
5. Django проверяет файл urls.py в своей структуре проекта, чтобы найти соответствующий URL-шаблон для этой страницы.
6. Когда Django находит соответствующий URL-шаблон, он вызывает соответствующую функцию представления (view function).
7. Функция представления обрабатывает запрос и выполняет все необходимые действия, такие как получение данных из базы данных или выполнение бизнес-логики.
8. Если данные нужны для отображения страницы, Django может использовать модели данных и базу данных для извлечения этих данных.
9. Далее Django использует шаблон HTML, который соответствует этой странице, чтобы создать окончательную HTML-страницу.
10. Django заполняет шаблон данными, которые были получены из базы данных или обработаны функцией представления.
11. Окончательная HTML-страница готова и возвращается обратно на сервер.
12. Сервер отправляет эту HTML-страницу обратно в браузер пользователя.
13. Браузер получает HTML-страницу и начинает ее отображать, показывая пользователю содержимое страницы, такое как текст, изображения и ссылки.

Вот таким образом происходит процесс от запроса до отрисовки страницы в Django!

Установка виртуальной среды

Виртуальная среда или **venv** не является неотъемлимой частью разработки на Django. Однако ее рекомендуется использовать, так как она позволяет создать множество виртуальных сред Python на одной операционной системе. Благодаря виртуальной среде приложение может запускаться независимо от других приложений на Python.

В принципе можно запускать приложения на Django и без виртуальной среды. В этом случае все пакеты Django устанавливаются глобально. Однако что если после создания первого приложения выйдет новая версия Django? Если мы захотим использовать для второго проекта новую версию Django, то из-за глобальной установки пакетов придется обновлять первый проект, который использует старую версию. Это потребует некоторой дополнительной работы по обновлению, так как не всегда соблюдается обратная совместимость между пакетами. Если мы решим использовать для второго проекта старую версию, то мы лишиемся потенциальных преимуществ новой версии. И использование виртуальной среды как раз позволяет разграничить пакеты для каждого проекта.

Для работы с виртуальной средой в python применяется встроенный модуль **venv**

Итак, создадим вируальную среду. Вначале определим каталог для проектов django. Например, пусть это будет каталог **C:\django**. Прежде всего перейдем в терминале/командной строке в этот каталог с помощью команды cd.

```
cd C:\django
```

Затем для создания виртуальной среды выполним следующую команду:

```
python3 -m venv .venv
```

Модулю venv передается название среды, которая в данном случае будет называться ".venv". Для наименования виртуальных сред нет каких-то определенных условностей. Пример консольного вывода:

```
C:\Users\eugen>cd C:\djangoПереход к папке будущей виртуальной среды
C:\django>python3 -m venv .venvСоздание виртуальной среды  C:\django>
```

После этого в текущей папке (C:\django) будет создан подкаталог ".venv".

Активация виртуальной среды

Для использования виртуальную среду надо активировать. И каждый раз, когда мы будем работать с проектом Django, связанную с ним виртуальную среду **надо активировать**. Например, активируем выше созданную среду, которая располагается в текущем каталоге в папке .venv. Процесс активации немного отличается в зависимости от операционной системы и от того, какие инструменты применяются. Так, в Windows можно использовать командную строку и PowerShell, но между ними есть отличия.

Активация в Windows в коммандной строке

Если наша ОС - Windows, то в папке **.venv/Scripts/** мы можем найти файл **activate.bat**), который активирует виртуальную среду. Так, в Windows активация виртуальной среды **в коммандной строке** будет выглядеть таким образом:

```
.venv\Scripts\activate.bat
```

Активация в Windows в PowerShell

Также при работе на Windows в папке **.venv/Scripts/** мы можем найти файл **activate.ps1**, который также активирует виртуальную среду, но применяется только в PowerShell. Но при работе с **PowerShell** следует учитывать, что по умолчанию в этой оболочке запрещено применять скрипты. Поэтому перед активацией среды необходимо установить разрешения для текущего

пользователя. Поэтому для активации виртуальной среды в PowerShell необходимо выполнить две следующих команды:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

.venv\Scripts\Activate.ps1
```

Установка Django

После активации виртуальной среды для установки Django выполним в консоли следующую команду

```
python3 -m pip install Django
```

Она устанавливает последнюю версию Django.

```
(.venv) C:\django>python -m pip install Django Collecting Django   Downloading
django-6.0-py3-none-any.whl.metadata (3.9 kB) Collecting asgiref>=3.9.1 (from
Django)   Downloading asgiref-3.11.0-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.5.0 (from Django)   Downloading sqlparse-0.5.4-py3-none-
any.whl.metadata (4.7 kB) Downloading django-6.0-py3-none-any.whl (8.3 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.3/8.3 MB 1.5 MB/s
eta 0:00:00 Downloading asgiref-3.11.0-py3-none-any.whl (24 kB) Downloading
sqlparse-0.5.4-py3-none-any.whl (45 kB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 45.9/45.9 kB 2.9
MB/s eta 0:00:00 Installing collected packages: sqlparse, asgiref, Django
Successfully installed Django-6.0 asgiref-3.11.0 sqlparse-0.5.4  (.venv) C:\django>
```

Если нам интересует конкретная версия Django, то мы можем указать ее при установке:

```
python3 -m pip install django~=5.0.0
```

Проверка установки

Чтобы убедиться, что все установлено правильно, мы можем перейти к интерпретатору python. Для этого введем в терминале команду

```
python3
```

И затем выполним последовательно следующие две инструкции:

```
>>> import django

>>> print(django.get_version())
```

Консольный вывод в моем случае:

```
(.venv) C:\django>python3

Python 3.12.3 (main, Nov  6 2025, 13:44:16) [GCC 13.3.0] on linux

Type "help", "copyright", "credits" or "license" for more information.

>>> import django

>>> print(django.get_version())

6.0

>>>
```

При установке Django в папке виртуальной среды устанавливается утилита **django-admin**. А на Windows также исполняемый файл **django-admin.exe**. Их можно найти в папке виртуальной среды, в которую производилась установка Django: на Windows - в подкаталоге **Scripts**.

django-admin предоставляет ряд команд для управления проектом Django. В частности, для создания проекта применяется команда **startproject**. Этой команде в качестве аргумента передается название проекта.

Итак, создадим первый на Django. Пусть он будет располагаться в той же папке, где располагается каталог виртуальной среды. И для этого вначале активируем ранее созданную виртуальную среду (например, среду .venv, которая была создана в прошлой теме, если она ранее не была активирована).

И после активации виртуальной среды выполним следующую команду

```
c:\django>django-admin startproject walrider_project
```

В данном случае мы создаем проект с именем "walrider_project". И после выполнения этой команды в текущей папке (c:\django) будет создан каталог.

Созданный каталог будет состоять из следующих элементов:

- **manage.py**: выполняет различные команды проекта, например, создает и запускает приложение
- **walrider_project**- собственно папка проекта metanit, которая содержит следующие файлы:
- **__init__.py**: данный файл указывает, что папка, в которой он находится, будет рассматриваться как модуль. Это стандартный файл для программы на языке Python.
- **settings.py**: содержит настройки конфигурации проекта
- **urls.py**: содержит шаблоны URL-адресов, по сути определяет систему маршрутизации проекта
- **wsgi.py**: содержит свойства конфигурации WSGI (Web Server Gateway Inerface). Он используется при развертывании проекта.
- **asgi.py**: название файла представляет сокращение от Asynchronous Server Gateway Interface и расширяет возможности WSGI, добавляя поддержку для взаимодействия между асинхронными веб-серверами и приложениями.

Запустим проект на выполнение. Для этого с помощью команды cd перейдем в консоли к папке проекта. И затем для запуска проекта выполним следующую команду:

python manage.py runserver

После запуска проекта в консоли мы увидим адрес, по которому запущен проект. Как правило, это адрес **http://127.0.0.1:8000/**. Откроем любой веб-браузер и введем данный адрес в адресную строку браузера. И нам откроется содержимое по умолчанию.

Веб-приложение или проект Django состоит из отдельных приложений. Вместе они образуют полноценное веб-приложение. Каждое приложение представляет какую-то определенную функциональность или группу функциональностей. Один проект может включать множество приложений. Это позволяет выделить группу задач в отдельный модуль и разрабатывать их относительно независимо от других. Кроме того, мы можем переносить приложение из одного проекта в другой независимо от другой функциональности проекта.

При создании проекта он уже содержит несколько приложений по умолчанию.

- **django.contrib.admin**
- **django.contrib.auth**
- **django.contrib.contenttypes**
- **django.contrib.sessions**
- **django.contrib.messages**
- **django.contrib.staticfiles**

Список всех приложений можно найти в проекте в файле **settings.py** в переменной **INSTALLED_APPS**:

```
INSTALLED_APPS = [
  'django.contrib.admin',
  'django.contrib.auth',
  'django.contrib.contenttypes',
  'django.contrib.sessions',
  'django.contrib.messages',
  'django.contrib.staticfiles',
]
```

И, конечно, мы можем создавать свои приложения, которые реализуют определенный функционал. Для создания приложения в проекте применяется команда

```
python manage.py startapp название_приложения
```

Так, создадим в проекте, созданном в прошлой теме или новом, первое приложение, которое будет называться **hello**. Для этого выполним в командной строке/терминале следующую команду:

```
python manage.py startapp hello
```

Рассмотрим вкратце его структуру:

- папка **migrations**: предназначена для хранения миграций - скриптов, которые позволяют синхронизировать структуру базы данных с определением моделей
- **__init__.py**: указывает интерпретатору python, что текущий каталог будет рассматриваться в качестве пакета
- **admin.py**: предназначен для административных функций, в частности, здесь призводится регистрация моделей, которые используются в интерфейсе администратора

- **apps.py**: определяет конфигурацию приложения
- **models.py**: хранит определение моделей, которые описывают используемые в приложении данные
- **tests.py**: хранит тесты приложения
- **views.py**: определяет функции, которые получают запросы пользователей, обрабатывают их и возвращают ответ

Но пока приложение никак не задействуется. Его надо зарегистрировать в проекте Django. Для этого откроем файл **settings.py** и добавим в конец массива INSTALLED_APPS наше приложение:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'hello',
]
```

В проекте может быть несколько приложений, и каждое из них надо добавлять таким вот образом.

Теперь определим какие-нибудь простейшие действия, которые будет выполнять данное приложение, например, отправлять в ответ пользователю строку "Hello World".

Для этого перейдем в проекте приложения **hello** к файлу **views.py**, который по умолчанию должен выглядеть следующим образом:

```
from django.shortcuts import render
# Create your views here.
```

Изменим код следующим образом:

```
1from django.http import HttpResponse
2
3def index(request):
4    return HttpResponse("Hello METANIT.COM")
```

В данном случае мы импортируем класс **HttpResponse** из стандартного пакета *django.http*. Затем определяется функция index(), которая в качестве параметра получает объект запроса request. Класс HttpResponse предназначен для создания ответа, который отправляется пользователю. И с помощью выражения return HttpResponse("Hello Bro") мы отправляем пользователю строку "Hello Bro"

Теперь также в основном проекте Django откроем файл **urls.py**, который позволяет сопоставить маршруты с представлениями, которые будут обрабатывать запрос по этим маршрутам. По умолчанию этот файл выглядит следующим образом:

```
1from django.contrib import admin
2from django.urls import path
3
4urlpatterns = [
5    path('admin/', admin.site.urls),
6]
```

Первой строкой из модуля *django.contrib* импортируется класс AdminSite, который предоставляет возможности работы с интерфейсом администратора. Второй строкой из модуля *django.urls* импортируется функция **path**. Эта функция задает сопоставление определенного маршрута с функцией обработки. Так, в данном случае маршрут "admin/" будет обрабатываться методом admin.site.urls.

Но выше мы определили функцию index в файле **views.py**, который возвращает пользователю строку "Hello Bro". Поэтому изменим файл **urls.py** следующим образом:

```
1from django.urls import path
2from hello import views
3
4urlpatterns = [
5    path('', views.index, name='home'),
6]
```

Чтобы использовать функцию views.index вначале импортируем модуль views. Затем определяем сопоставление маршрута ' ' и функции views.index и также дополнительно имя для маршрута: name='home'. По сути маршрут ' ' будет сопоставляться с запросом к корню приложения.

Представления и маршрутизация

Центральным моментом любого веб-приложения является обработка запроса, который отправляет пользователь. В Django за обработку запроса отвечают представления или **views**. По сути представления представляют функции обработки, которые принимают данные запроса в виде объекта **HttpRequest** из пакета django.http и генерируют некоторый результат, который затем отправляется пользователю.

По умолчанию представления размещаются в приложении в файле **views.py**.

При создании нового проекта файл **views.py** имеет следующее содержимое:

```
1 from django.shortcuts import render
2 # Create your views here.
```

Данный код пока никак не обрабатывает запросы, он только импортирует функцию **render()**, которая может использоваться для обработки.

Генерировать результат можно различными способами. Один из них представляет использование класса **HttpResponse** из пакета django.http, который позволяет отправить текстовое содержимое.

Так, изменим файл **views.py** следующим образом:

```
1 from django.http import HttpResponse
2
3 def index(request):
4     return HttpResponse("Главная")
5
6 def about(request):
7     return HttpResponse("О сайте")
8
9 def contact(request):
10    return HttpResponse("Контакты")
```

В данном случае определены три функции, которые будут обрабатывать запросы. Каждая функция принимает в качестве параметра request объект **HttpRequest**, который хранит информацию о запросе. Однако в данном случае она нам не нужны, поэтому параметр никак не используется. Для генерации ответа в конструктор объекта HttpResponse передается некоторая строка. Это может быть в том числе и код html в виде строки.

Чтобы эти функции сопоставлялись с запросами, надо определить для них маршруты в проекте в файле **urls.py**. В частности, изменим этот файл следующим образом:

```
1from django.urls import path
2from hello import views
3
4urlpatterns = [
5    path('', views.index),
6    path('about', views.about),
7    path('contact', views.contact),
8]
```

Переменная **urlpatterns** определяет набор сопоставлений функций обработки с определенными строками запроса. Например, запрос к корню веб-сайта будет обрабатываться функцией index, запрос по адресу "about" будет обрабатываться функцией about, а запрос "contact" - функцией contact.

Запустим проект и обратимся по некоторым из этих адресов.

При этом мы можем отправлять не простой текст, а, например, код html, который затем интерпретируется браузером. Так, изменим файл **views.py** следующим образом:

```
1from django.http import HttpResponse
2
3def index(request):
```

```
4    return HttpResponse("<h2>Главная</h2>")
5
6def about(request):
7    return HttpResponse("<h2>О сайте</h2>")
8
9def contact(request):
10    return HttpResponse("<h2>Контакты</h2>")
```

Это так называемые функции-представления или view function. И в файле **urls.py** проекта они сопоставляются с адресами URL с помощью функции **path()**:

```
1from django.urls import path
2from hello import views
3
4urlpatterns = [
5    path('', views.index),
6    path('about', views.about),
7    path('contact', views.contact),
8]
```

За сопоставление путей и функций-представлений отвечает функция **path()**, которая располагается в пакете **django.urls** и которая принимает четыре параметра:

```
1path(route, view, kwargs=None, name=None)
```

- **route**: представляет шаблон адреса URL, которому должен соответствовать запрос
- **view**: функция-представление, которое обрабатывает запрос
- **kwargs**: дополнительные аргументы, которые передаются в функцию-представление
- **name**: название маршрута

В примере выше применялись только первые два параметра, которые являются обязательными: запрошенный адрес URL и функция, которая обрабатывает запрос по этому адресу. Дополнительно через третий параметр можно указать имя маршрута:

```
1path(', views.index, name='home'),
```

В данном случае маршрут будет называться "home".

re_path

Хотя мы можем успешно применять функцию **path()** для определения маршрутов, она довольно ограничена по своему действию. Запрошенный путь должен в точности соответствовать указанному в маршруте адресу URL. Так, в примере выше чтобы функция views.about могла обрабатывать запрос, адрес должен быть в точности "about". Например, стоит нам указать слеш в конце: "about/" и django уже не сможет сопоставить путь с запросом.

В качестве альтернативы для определения маршрутов мы можем использовать функцию **re_path()**, которая также располагается в пакете **django.urls** и имеет тот же набор параметров:

```
1re_path(route, view, kwargs=None, name=None)
```

Ее преимущесто состоит в том, что она позволяет задать адреса URL с помощью регулярных выражений.

Например, изменим определение файла **urls.py** следующим образом:

```
1from django.urls import path, re_path
2from hello import views
3
4urlpatterns = [
5    path(', views.index),
6    re_path(r'^about', views.about),
7    re_path(r'^contact', views.contact),
```

```
8]
```

Адрес в первом маршруте по-прежнему образуется с помощью функции path и указывает на корень веб-приложения.

Остальные два маршрута образуются с помощью функции re_path(). Причем, поскольку определяется регуляное выражение, то перед строкой с шаблоном адреса URL ставится буква **r**. В самом шаблоне адреса можно использовать различные элементы синтаксиса регулярных выражений. В частности, выражение ^about указывает, что адрес должен начинаться с "about". Однако он необязательно в точности должен соответствовать строке "about", как это было в случае с функцией path.

Например, мы можем обратиться по любому адресу, главное чтобы он начинался с "about", и тогда подобный запрос будет обрабатываться функцией views.about.

Очередность маршрутов

Когда запрос приходит к приложению, то система проверяет соответствие запроса маршрутам по мере их определения: вначале сравнивается первый маршрут, если он не подходит, то сравнивается второй и так далее. Поэтому более общие маршруты должны определяться в последнюю очередь, а более конкретные маршруты должны идти в начале. Например:

```
1from django.urls import path, re_path
2from hello import views
3
4urlpatterns = [
5    re_path(r'^about/contact/', views.contact),
6    re_path(r'^about', views.about),
7    path('', views.index),
8]
```

В данном случае адрес "^about/contact" представляет более конкретный маршрут по сравнению с "^about". Поэтому он определяется в первую очередь.

Если бы было наоборот:

```
urlpatterns = [
    path('', views.index),
    re_path(r'^about', views.about),
    re_path(r'^about/contact', views.contact),
]
```

то запрос по адресу "about/contact" обрабатывался бы функцией views.about

Основные элементы синтаксиса регуляных выражений

Некоторые базовые элементы регуляных выражений, которые можно использовать для определения адресов URL:

- ^(начало адреса)
- $(конец адреса)
- +(1 и более символов)
- ?(0 или 1 символ)
- {n}(n символов)
- {n, m}(от n до m символов)
- .(любой символ)
- \d+(одна или несколько цифр)
- \D+(одна или несколько НЕ цифр)
- \w+(один или несколько буквенных символов)

Передача значений в функцию

Выше были рассмотрены все параметры функций path и re_path, кроме одного - kwargs, который позволяет передать в функцию-представление некоторые значения. Например, в файле **views.py** определим следующие функции:

```
1 from django.http import HttpResponse
2
3 def index(request):
4     return HttpResponse("<h2>Главная</h2>")
5
6 def about(request, name, age):
7     return HttpResponse(f"""
8             <h2>О пользователе</h2>
9             <p>Имя: {name}</p>
10            <p>Возраст: {age}</p>
11    """)
```

Здесь функция about() также принимает два дополнительных параметра: name и age (условно имя и возраст пользователя). В функции их значения отправляются пользователю вместе с остальным содержимым.

Изменим файл **urls.py**:

```
1 from django.urls import path
2 from hello import views
3
4 urlpatterns = [
5     path('', views.index),
6     path('about', views.about, kwargs={"name":"Tom", "age": 38}),
7 ]
```

С помощью параметра kwargs в функцию about передается словарь с двумя значениями - для двух параметров функции.

Получение данных запроса. HttpRequest

Функции-представления в качестве обязательного параметра получают объект **HttpRequest**, который хранит информацию о запросе. HttpRequest

определяет ряд атрибутов, которые хранят информацию о запросе. Выделим следующие из них:

- **scheme**: схема запроса (http или https)
- **body**: представляет тело запроса в виде строки байтов
- **path**: представляет путь запроса
- **method**: метод запроса (GET, POST, PUT и т.д.)
- **encoding**: кодировка
- **content_type**: тип содержимого запроса (значение заголовка CONTENT_TYPE)
- **GET**: объект в виде словаря, который содержит параметры запроса GET
- **POST**: объект в виде словаря, который содержит параметры запроса POST
- **COOKIES**: отправленные клиентом куки
- **FILES**: отправленные клиентом файлы
- **META**: хранит все доступные заголовки http в виде словаря. Набор заголовков зависит от клиента и сервера, некоторые из них:
- CONTENT_LENGTH: длина содержимого.
- CONTENT_TYPE: MIME-тип запроса.
- HTTP_ACCEPT: типы ответа, которые принимает клиент.
- HTTP_ACCEPT_ENCODING: кодировка, в которой клиент принимает ответ.
- HTTP_ACCEPT_LANGUAGE: язык ответа, который принимает клиент.
- HTTP_HOST: хост сервера.
- HTTP_REFERER: страница, с которой клиент отправил запрос (при ее наличии).
- HTTP_USER_AGENT: юзер-агент или информация о браузере клиента.
- QUERY_STRING: строка запроса.
- REMOTE_ADDR: IP-адрес клиента.
- REMOTE_HOST: имя хоста клиента.

- REMOTE_USER: аутентификационные данные клиента (при наличии)
- REQUEST_METHOD: тип запроса (GET, POST).
- SERVER_NAME: имя хоста сервера.
- SERVER_PORT: порт сервера.
- **headers**: заголовки запроса в виде словаря

Также HttpRequest определяет ряд методов. Отметим следующие из них:

- **get_full_path()**: возвращает полный путь запроса, включая строку запроса
- **get_host()**: возвращает хост клиента, для этого используется значения заголовков HTTP_X_FORWARDED_HOST (если включена опция USE_X_FORWARDED_HOST) и HTTP_HOST
- **get_port()**: возвращает номер порта

Например, получим некоторую информацию о запросе. Для этого в файле **views.py**:

```
1 from django.http import HttpResponse
2
3 def index(request):
4     host = request.META["HTTP_HOST"] # получаем адрес сервера
5     user_agent = request.META["HTTP_USER_AGENT"]    # получаем
6 данные бразера
7     path = request.path     # получаем запрошенный путь
8
9     return HttpResponse(f"""
10        <p>Host: {host}</p>
11        <p>Path: {path}</p>
12        <p>User-agent: {user_agent}</p>
      """)
```

В данном случае получаем два заголовка "HTTP_HOST" и "HTTP_USER_AGENT" и запрошенный путь.

В файле **urls.py** зарегистрируем данную функцию:

```
1from django.urls import path
2from hello import views
3
4urlpatterns = [
5    path("index", views.index),
6]
```

HttpResponse и отправка ответа

Для отправки ответа клиенту в Django применяется класс **HttpResponse** из пакета **django.http**. В общем случае для отправки некоторых данных достаточно эти данные передать в конструктор HttpResponse. Например, пусть в файле **views.py** имеется простейшая функция-представление, которая отправляет ответ клиенту:

```
1from django.http import HttpResponse
2
3def index(request):
4    return HttpResponse("Hello")
```

И в файле **urls.py** эта функция соотносится с некоторым маршрутом:

```
1from django.urls import path
2from hello import views
3
4urlpatterns = [
5    path("", views.index),
6]
```

Подобная функция просто передает в HttpResponse некоторый текст, который пользователь затем увидит в браузере. Однако подобной функциональностью HttpResponse не ограничивается. Так, функция инициализации класса определяет несколько параметров:

```
HttpResponse.__init__(content=b'', content_type=None, status=200, reason=None, charset=None, headers=None)
```

Параметры:

- **content**: содержимое ответа в виде строки байтов. Если передается другое содержимое, то оно конвертируется в строку байтов
- **content_type**: MIME-тип ответа, устанавливает HTTP-заголовок Content-Type. Если этот параметр не установлен, то применяется mime-тип text/html и значение настройки DEFAULT_CHARSET, то есть в итоге будет: "text/html; charset=utf-8".
- **charset**: кодировка ответа в виде строки. По умолчанию django пытается установить кодировку из параметра content_type, а в случае неудачи для установки кодировки применяется настройка DEFAULT_CHARSET.
- **status**: статусный код ответа. По умолчанию равно 200
- **reason_phrase**: сообщение, которое отправляется в вместе статусным кодом
- **headers**: заголовки ответа в виде словаря

Для хранения отправляемых данных он определяет ряд атрибутов. Некоторые из них:

- **content**: содержимое ответа в виде строки байтов
- **headers**: отправляемые заголовки в виде словаря
- **charset**: кодировка ответа в виде строки. По умолчанию django пытается установить кодировку из заголовка content_type, а в случае

неудачи для установки кодировки применяется настройка DEFAULT_CHARSET.

- **status_code**: статусный код ответа
- **reason_phrase**: сообщение, которое отправляется в вместе статусным кодом

Рассмотрим некоторые возможности. Например, изменим определение функции в файле **views.py**:

```
1 from django.http import HttpResponse
2
3 def index(request):
4     return HttpResponse("Hello Bro", headers={"SecretCode": "21234567"})
```

В данном случае также устанавливается заголовок "SecretCode". Хотя в реальности в HTTP не существует такого заголовка, но мы можем определять кастомные заголовки, чтобы передать через них клиенту какую-нибудь информацию. Например, после обращения к функции мы можем в браузере через инструменты разработчика проинспектировать отправленные сервером заголовки и найти там в том числе заголовок "SecretCode".

Подобным образом можно установить другие параметры, например, статусный код и сообщение к нему:

```
1 from django.http import HttpResponse
2
3 def index(request):
4     return HttpResponse("Произошла ошибка", status=400, reason="Incorrect data")
```

Установка содержимого и кодировки:

```
1 from django.http import HttpResponse
2
3 def index(request):
```

```
4   return HttpResponse("<h1>Hello</h1>", content_type="text/plain",
charset="utf-8")
```

Хотя в содержимом ответа применяются теги html (<h1>), но браузере теперь будет рассматривать это содержимое как простой текст, потому что установлен заголовок "text/plain"

Задание 1

В views.py приложения posts создайте функции:

index - возвращает "Страница блога"

post_detail – принимает post_id и возвращает "Информация о посте с ID: {id}"

about – возвращает HTML с заголовком и текстом о себе

В urls.py проекта пропишите маршруты для этих функций. В функцию about добавьте вывод IP-адреса клиента и типа браузера из объекта request.

Функции-представления могут принимать параметры, через которые могут передаваться различные данные. Подобные параметры передаются в адресе URL. Например, в запросе

```
1http://127.0.0.1:8000/index/Tom/38/
```

последние два сегмента Tom/38/ могут представлять параметры URL, которые могут быть связанны с параметрами функции-представления через систему маршрутизации. Подобные параметры еще можно назвать **параметрами маршрута**

Определение параметров через функцию path

Определим в файле **views.py** следующие функции:

```
1from django.http import HttpResponse
2
3def index(request):
4    return HttpResponse("<h2>Главная</h2>")
5
6def user(request, name):
7    return HttpResponse(f"<h2>Имя: {name}</h2>")
```

В данном случае функция user имеет два параметра. Для второго параметра - name мы будем получать данные из строки запроса. То есть это будет параметр маршрута. И как любой другой параметр мы сможем использовать его внутри функции.

Далее в файле **urls.py** определим следующий код:

```
1from django.urls import path
2from hello import views
3
4urlpatterns = [
5    path("", views.index),
```

```
6    path("user/<str:name>", views.user),
7]
```

Для второго маршрута здесь определен параметр name (который соответствует параметру name функции views.user). Параметры заключаются в угловые скобки в формате <спецификатор:название_параметра>. Например, здесь параметр name имеет спецификатор **str**.

По умолчанию Django предоставляет следующие спецификаторы:

- **str**: соответствует любой строке за исключенем символа "/". Если спецификатор не указан, то используется по умолчанию
- **int**: соответствует любому положительному числу
- **slug**: соответствует последовательности буквенных символов ASCII, цифр, дефиса и символа подчеркивания, например, building-your-1st-django-site
- **uuid**: сооветствует идентификатору UUID, например, 075194d3-6885-417e-a8a8-6c931e272f00
- **path**: соответствует любой строке, которая также может включать символ "/" в отличие от спецификатора str

Также отмечу, что количество и название параметров в шаблонах адресов URL соответствуют количеству и названиям параметров соответствующих функций, которые обрабатывают запросы по данным адресам.

Запустим приложение, обратимся к функции views.user, например, с помошью запроса *http://127.0.0.1:8000/user/Tom*, и через строку запроса - через ее третий сегмент мы сможем передать значение для параметра name

Подобным образом можно определить и большее количество параметров. Например, добавим второй параметр в функцию user в **views.py**:

```
1from django.http import HttpResponse
2
```

```
3def index(request):
4    return HttpResponse("<h2>Главная</h2>")
5
6def user(request, name, age):
7    return HttpResponse(f"<h2>Имя: {name}  Возраст:{age}</h2>")
```

А в файле **urls.py** добавим в маршрут параметр age:

```
1from django.urls import path
2from hello import views
3
4urlpatterns = [
5    path("", views.index),
6    path("user/<name>/<int:age>", views.user),
7]
```

Предполагается, что параметр age будет представлять число, поэтому для него используется спецификатор **int**. В этом случае мы можем обратиться к функции user, например, с помощью запроса

```
1http://127.0.0.1:8000/user/Tom/38
```

В этом случае сегмент "/Tom/" будет сопоставлен с параметром name, а "/38" - с параметром age.

Значения для параметров по умолчанию

В примере выше использовались два параметра, но что, если мы не передадим для одного или обоих параметров значения?

В этом случае мы получим ошибку. Параметры маршрута являются частью шаблона URL. Соответственно если в строке запроса не передаются значения для параметров, такой запрос не соответствует маршруту. Поэтому Django не сможет найти нужный маршрут для обработки запроса, и мы получим ошибку 404 (ресурс не найден).

Однако такое поведение не всегда может быть желательным. И мы можем задать для параметров маршрута значения по умолчанию на случай, если через строку запроса не передаются значения. Так, для функции user в **views.py** определим значения для параметров по умолчанию:

```
1from django.http import HttpResponse
2
3def index(request):
4    return HttpResponse("<h2>Главная</h2>")
5
6def user(request, name="Undefined", age =0):
7    return HttpResponse(f"<h2>Имя: {name}  Возраст: {age}</h2>")
```

В данном случае, если для параметра name не передается значение, то он получает в качестве значения строку "Undefined". Для параметра age значение по умолчанию 0.

В этом случае для функции user в файле **urls.py** надо определить дополнительные маршруты, которые не учитывают необязательные параметры:

```
1from django.urls import path
2from hello import views
3
4urlpatterns = [
5    path("", views.index),
6    path("user", views.user),
7    path("user/<name>", views.user),
8    path("user/<name>/<int:age>", views.user),
9]
```

Определение параметров через функцию re_path

Подобным образом мы можем использовать функцию **re_path** для определения параметров. Определим в приложении в файле **views.py** следующие функции:

```
1from django.http import HttpResponse
2
3def index(request):
4    return HttpResponse("<h2>Главная</h2>")
5
6def user(request, name, age):
7    return HttpResponse(f"<h2>Имя: {name}  Возраст: {age}</h2>")
```

Теперь изменим файл **urls.py**, чтобы он мог сопоставить данные функции с запросами:

```
1from django.urls import path, re_path
2from hello import views
3
4urlpatterns = [
5    path("", views.index),
6    re_path(r"^user/(?P<name>\D+)/(?P<age>\d+)", views.user),
7]
```

Для представления параметра в шаблоне адреса используется выражение ?P<>. Общее определение параметру соответствует формату (?P<имя_параметра>регулярное_выражение). Между угловыми скобками помещается название параметра. После закрывающей угловой скобки идет регулярное выражение, которому дожно соответствовать значение параметра.

Во втором шаблоне адреса определяются два параметра: name и age. При этом параметр age должен представлять число, а параметр name должен состоять только из буквенных символов.

Также мы можем указать для определенных параметров значения по умолчанию:

```
1def user(request, name="Undefined", age =0):
2    return HttpResponse(f"<h2>Имя: {name}  Возраст: {age}</h2>")
```

В этом случае надо дополнительно определить еще маршруты в файле **urls.py** для тех запросов, в которых не передаются значения для маршрутов:

```
1from django.urls import path, re_path
2from hello import views
3
4urlpatterns = [
5    path("", views.index),
6    re_path(r"^user/(?P<name>\D+)/(?P<age>\d+)", views.user),
7    re_path(r"^user/(?P<name>\D+)", views.user),
8    re_path(r"^user", views.user),
9]
```

Обратите внимание на порядок размещения маршрутов: в отличие от случая с функцией path теперь сначала размещаются более конкретные маршруты с большим количеством параметров.

Вложенные маршруты и функция include

Функция **include()** позволяет определить вложенные маршруты или подмаршруты для некоторого маршрута. В качестве параметра она принимает набор маршрутов:

```
1include(pattern_list)
```

Параметр pattern_list представляет набор вызовов функций path() и/или re_path(). Например, определеим в файле **views.py** следующие функции:

```
1 from django.http import HttpResponse
2
3 def index(request):
4     return HttpResponse("Главная страница")
5
6 def products(request):
7     return HttpResponse("Список товаров")
8
9 def new(request):
10     return HttpResponse("Новые товары")
11
12 def top(request):
13     return HttpResponse("Наиболее популярные товары")
```

Здесь последние три функции функционально относятся к товарам.

И определим в файле **urls.py** следующие маршруты:

```
1 from django.urls import path, include
2 from hello import views
3
4 product_patterns = [
5     path("", views.products),
6     path("new", views.new),
7     path("top", views.top),
8 ]
9
10 urlpatterns = [
11     path("", views.index),
12     path("products/", include(product_patterns)),
13 ]
```

Здесь в виде переменной product_patterns отдельно определен набор маршрутов, который касается товаров.

Для установки этих маршрутов этот список передается в функцию include()

```
path("products", include(product_patterns)),
```

Причем этот список будет ассоциирован с шаблоном "product". В этом случае шаблоны вложенных маршрутов будут объединены с шаблоном родительского маршрута, и таким образом будет сформирован общий шаблон, которому должен соответствовать запрос.

Например, если придет запрос "http://127.0.0.1:8000/products/top", то он будет обрабатываться функцией top.

Таким образом, мы можем сгруппировать маршруты для запросов, которые начинаются с определенного шаблона. Что также позволяет избежать повтора частей шаблона, когда шаблон url начинается с одного и того же сегмента.

Получение параметров

Вложенные маршруты получают параметры, определенные в родительских маршрутах. Например, определим в файле **views.py** следующие функции-представления:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("Главная страница")

def products(request, id):
    return HttpResponse(f"Товар {id}")

def comments(request, id):
```

```
10    return HttpResponse(f"Комментарии о товаре {id}")
11
12def questions(request, id):
13    return HttpResponse(f"Вопросы о товаре {id}")
```

Здесь все функции кроме первой имеют второй параметр - id - условный номер товара.

Изменим файл **urls.py**:

```
 1from django.urls import path, include
 2from hello import views
 3
 4product_patterns = [
 5    path("", views.products),
 6    path("comments", views.comments),
 7    path("questions", views.questions),
 8]
 9
10urlpatterns = [
11    path("", views.index),
12    path("products/<int:id>/", include(product_patterns)),
13]
```

Здесь для второго маршрута определяется числовой параметр id. Этот параметр передается всем вложенным маршрутам, соответственно нам не надо определять данный параметр во вложенных маршрутах

От параметров, которые передаются через адрес URL, следует отличать параметры, которые передаются через строку запроса. Например, в запросе

```
1http://127.0.0.1:8000/index/3/Tom/
```

два последних сегмента - 3/Tom/ представляют параметры URL или параметры маршрута. А в запросе

```
http://127.0.0.1:8000/index?id=3&name=Tom
```

те же самые значения 3 и Tom представляют **параметры строки запроса**.

Параметры строки запроса указывается после вопросительного знака ?. Каждый параметр предсталяет пару ключ-значение, напимер, в id=3 : id - название или ключ параметра, а 3 - его значение. Параметры в строке запроса отделяются друг от друга знаком амперсанда.

Для получения параметров из строки запроса применяется метод **request.GET.get()**, в которую передается название параметра.

Например, определим в файле **views.py** следующие функции:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("<h2>Главная</h2>")

def user(request):
    age = request.GET.get("age")
    name = request.GET.get("name")
    return HttpResponse(f"<h2>Имя: {name}  Возраст: {age}</h2>")
```

Функция user принимает извлекает из строки запроса два параметра: name и age.

В файле **urls.py** определим следующие маршруты:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("", views.index),
    path("user/", views.user)
```

```
]
```

При обращении к приложению по адресу *http://127.0.0.1:8000/user/?name=Tom&age=22*, параметр name будет иметь значение "Tom", а параметр age - 22.

Однако возможна ситуация, когда для каких-то параметров не будет передано значение. В этом случае мы можем указать для подобных параметров значение по умолчанию. Для этого изменим определение функции user в **views.py**:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("<h2>Главная</h2>")

def user(request):
    age = request.GET.get("age", 0)
    name = request.GET.get("name", "Undefined")
    return HttpResponse(f"<h2>Имя: {name}  Возраст: {age}</h2>")
```

Второй параметр функции request.GET.get() представляет значение по умолчанию для параметра, для которого не задано значение.

Задание 2

Создайте представление posts_by_category, которое принимает параметр category и возвращает строку с названием категории. Пропишите маршрут category/<str:category>/.

Создайте представление search, которое извлекает из GET-параметров q и page (значение по умолчанию для page = 1) и возвращает строку с результатами поиска. Пропишите маршрут search/.

Переадресация

При перемещении документа с одного адреса на другой мы можем воспользоваться механизмом переадресации, чтобы указать пользователям и поисковику, что документу теперь доступен по новому адресу.

Переадресация бывает временная и постоянная. При временной переадресации мы указываем, что документ временно перемещен на новый адрес. В этом случае в ответ отправляется статусный код 302. При постоянной переадресации мы уведомляем, что документ теперь постоянно будет достуен по новому адресу

Для создания временной переадресации применяется класс **HttpResponseRedirect**, а для постоянной - класс **HttpResponsePermanentRedirect**, которые расположены в пакете django.http.

Так, определим в файле **views.py** следующий код:

```
1 from django.http import HttpResponse, HttpResponseRedirect,
2 HttpResponsePermanentRedirect
3
4 def index(request):
5     return HttpResponse("Index")
6
7 def about(request):
8     return HttpResponse("About")
9
10 def contact(request):
11     return HttpResponseRedirect("/about")
12
```

```
13def details(request):
    return HttpResponsePermanentRedirect("/")
```

При обращении к функции contact она будет перенаправлять по пути "about", который будет обрабатываться функцией about. А функция details будет использовать постоянную переадресацию и перенаправлять на корень веб-приложения.

И также в файле **urls.py** для тестирования определим следующие маршруты:

```
1from django.urls import path
2from hello import views
3
4urlpatterns = [
5    path("", views.index),
6    path("about/", views.about),
7    path("contact/", views.contact),
8    path("details/", views.details),
9]
```

Отправка статусных кодов

Также в пакете **django.http** есть ряд классов, которые позволяют отправлять определенный статусный код:

Статусный код	Класс
304 (Not Modified)	HttpResponseNotModified
400 (Bad Request)	HttpResponseBadRequest
403 (Forbidden)	HttpResponseForbidden
404 (Not Found)	HttpResponseNotFound
405 (Method Not Allowed)	HttpResponseNotAllowed
410 (Gone)	HttpResponseGone

500 (Internal Server Error)	HttpResponseServerError

В функцию конструктора этих классов можно передать некоторые данные, например, сообщение об ошибке, которое увидит пользователь:

```
1HttpResponseNotModified()
2HttpResponseBadRequest("Bad Request")
3HttpResponseForbidden("Forbidden")
4HttpResponseNotFound("Not Found")
5HttpResponseNotAllowed("Method is not allowed")
6HttpResponseGone("Content is no longer here")
7HttpResponseServerError("Server Error")
```

Например, определим следующий файл **views.py**:

```
1from django.http import HttpResponse, HttpResponseNotFound,
2HttpResponseForbidden, HttpResponseBadRequest
3
4
5def index(request, id):
6    people = ["Tom", "Bob", "Sam"]
7    # если пользователь найден, возвращаем его
8    if id in range(0, len(people)):
9        return HttpResponse(people[id])
10   # если нет, то возвращаем ошибку 404
11   else:
12       return HttpResponseNotFound("Not Found")
13
14def access(request, age):
15   # если возраст НЕ входит в диапазон 1-110, посылаем ошибку 400
16   if age not in range(1, 111):
17       return HttpResponseBadRequest("Некорректные данные")
```

```
18    # если возраст больше 17, то доступ разрешен
19    if(age > 17):
20        return HttpResponse("Доступ разрешен")
21    # если нет, то возвращаем ошибку 403
22    else:
        return HttpResponseForbidden("Доступ заблокирован: недостаточно
лет")
```

Функция index принимает параметр id. Это будет индекс элемента в списке people. Однако пользователь может передать и недействительный индекс, например, 100, хотя в примере в списке только 3 элемента. Поэтому, если передан действительный индекс, то возвращаем элемент по этому индексу. Если же индекс нейдествителен, то с помощью класса HttpResponseNotFound возвращаем ошибку 404 и сообщение об ошибке.

Аналогично функция access принимает параметр age, который представляет условный возраст пользователя. Если возраст выходит за некоторые разумные пределы (1-110), то с помощью класса HttpResponseBadRequest возвращаем ошибку 400. Если возраст укладывается в эти рамки, но он меньше 18, то с помощью класса HttpResponseForbidden возвращаем ошибку 403 о том, что доступ запрещен.

Для теста в файле **urls.py** определим следующие маршруты:

```
1from django.urls import path
2from hello import views
3
4urlpatterns = [
5    path("index/<int:id>", views.index),
6    path("access/<int:age>", views.access),
7]
```

Обратися к функции index, передав для параметра id корректные и некорректные значения.

Отправка json

За отправку клиенту данных в формате JSON в Django отвечает специальный класс - **JsonResponse**, который по сути представляет подкласс HttpResponse. Основная особенность **JsonResponse** состоит в том, что при отправке данных он автоматически устанавливает для заголовка **Content-Type** (тип содержимого) значение **application/json**

Его конструктор принимает ряд параметров:

```
def __init__(data, encoder=DjangoJSONEncoder, safe=True, json_dumps_params=None, **kwargs)
```

- **data**: отправляемые данные
- **encoder**: сериализатор, которые сериализует отправляемые данные в формат JSON. По умолчанию представляет тип django.core.serializers.json.DjangoJSONEncoder
- **safe**: представляет булевое значение. Если равно False, то сериализации подлежит любой объект. Если же равно True, то отправляемые данные должны представлять тип dict - то есть словарь. По умолчанию равно True
- **json_dumps_params**: словарь аргументов, который передается в функцию json.dumps() для генерации ответа

Например, отправим какие-нибудь данные в формате JSON. Для этого определим в файле **views.py** следующий код:

```
from django.http import JsonResponse

def index(request):
    return JsonResponse({"name": "Tom", "age": 38})
```

Сериализация произвольных объектов

По умолчанию JsonResponse сериазует и отправляет только словари. Однако что, если мы хотим отправить объект какого-то своего типа? В этом случае

неоьходимо определить класс-сериализатор, который будет содержать логику сериализации объекта в json. Например:

```
1 from django.http import JsonResponse
2 from django.core.serializers.json import DjangoJSONEncoder
3
4 def index(request):
5     bob = Person("Bob", 41)
6     return JsonResponse(bob, safe=False, encoder=PersonEncoder)
7
8 class Person:
9
10     def __init__(self, name, age):
11         self.name = name    # имя человека
12         self.age = age       # возраст человека
13
14 class PersonEncoder(DjangoJSONEncoder):
15     def default(self, obj):
16         if isinstance(obj, Person):
17             return {"name": obj.name, "age": obj.age}
18             # return obj.__dict__
19         return super().default(obj)
```

В данном случае JsonResponse отправляет объект типа Person, у которого определены два атрибута: name и age.

Объект Person не является словарем, поэтому параметр safe имеет значение False. Кроме того, параметр encoder указывает на сериализатор, который будет сериализовать данные в json. В данном случае это класс PersonEncoder.

Класс сериализатора наследуется от django.core.serializers.json.DjangoJSONEncoder. Он реализует метод default,

который возвращает сериализованный объект. В частности, в этом методе сначала проверяем, представляет ли параметр объект Person. И если представляет, то возвращаем словарь из значений атрибутов объекта

```
return {"name": obj.name, "age": obj.age}
```

Стоит отметить, что в данном случае мы можем просто возвратить представление объекта в виде словаря

```
return obj.__dict__
```

Однако в отдельных ситуациях может потребоваться более тонкая настройка сериализации.

Если же объект не представляет тип Person, то передаем его в реализацию метода default родительского класса.